

Flutter CustomPainter Boilerplate Documentation

Overview

`CustomPainter` is a powerful Flutter class that allows you to draw custom shapes, graphics, and designs directly onto the screen using a `Canvas`.

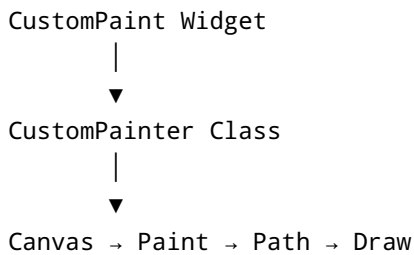
This boilerplate provides a clean and reusable starting point for drawing custom UI elements such as:

- Custom shapes
- Waves
- Charts
- Background designs
- Animations
- Game graphics

Architecture Overview

This boilerplate consists of two main parts:

1. **Custom Painter Class** → Handles drawing logic
2. **CustomPaint Widget** → Displays the drawing in the UI



Part 1: Custom Painter Class

```
class MyCustomPainter extends CustomPainter
```

This class is responsible for drawing on the canvas.

Method 1: paint()

```
@override  
void paint(Canvas canvas, Size size)
```

This method is called whenever Flutter needs to draw.

Parameters:

- `Canvas canvas` → Surface to draw on
 - `Size size` → Size of the available drawing area
-

Step 1: Create Paint (Brush)

```
final paint = Paint()  
  ..color = Colors.blue  
  ..style = PaintingStyle.fill  
  ..strokeWidth = 2;
```

Paint controls:

- Color
- Stroke width
- Fill or stroke
- Opacity
- Gradients
- Shader

Examples:

```
..style = PaintingStyle.stroke // outline only  
..strokeWidth = 5  
..color = Colors.red.withOpacity(0.5)
```

Step 2: Create Path (Shape)

```
final path = Path();  
path.moveTo(0, size.height);  
path.lineTo(size.width / 2, 0);
```

```
path.lineTo(size.width, size.height);  
path.close();
```

Path is used to define shapes.

Common methods:

- moveTo(x, y)
- lineTo(x, y)
- quadraticBezierTo()
- cubicTo()
- arcTo()
- close()

This example creates a triangle.

Step 3: Draw on Canvas

```
canvas.drawPath(path, paint);
```

Canvas drawing methods:

- drawPath()
 - drawRect()
 - drawCircle()
 - drawLine()
 - drawOval()
 - drawRRect()
-

Method 2: shouldRepaint()

```
@Override  
bool shouldRepaint(covariant CustomPainter oldDelegate) {  
    return false;  
}
```

Controls when repaint happens.

Use:

- `false` → Static drawing (better performance)
- `true` → Dynamic drawing (animations, state change)

Part 2: CustomPaint Widget

```
CustomPaint(  
  size: Size(300, 200),  
  painter: MyCustomPainter(),  
)
```

This widget displays the painter.

Important properties:

- `painter` → Foreground painter
- `foregroundPainter` → Paint above child
- `size` → Drawing area size
- `child` → Widget inside

Full Boilerplate Example (Recommended Template)

```
import 'package:flutter/material.dart';  
  
class MyCustomPainter extends CustomPainter {  
  @override  
  void paint(Canvas canvas, Size size) {  
    final paint = Paint()  
      ..color = Colors.blue  
      ..style = PaintingStyle.fill  
      ..strokeWidth = 2;  
  
    final path = Path();  
  
    path.moveTo(0, size.height);  
    path.lineTo(size.width / 2, 0);  
    path.lineTo(size.width, size.height);  
    path.close();  
  
    canvas.drawPath(path, paint);  
  }  
  
  @override  
  bool shouldRepaint(covariant CustomPainter oldDelegate) {  
    return false;  
  }  
}
```

```

}

class CustomPaintExample extends StatelessWidget {
  const CustomPaintExample({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: CustomPaint(
          size: const Size(300, 200),
          painter: MyCustomPainter(),
        ),
      ),
    );
  }
}

```

Coordinate System Explanation

Flutter canvas uses this coordinate system:

(0,0) → Top Left

X increases → Right

Y increases → Down

Example:

Top Left	(0,0)
Top Right	(width,0)
Bottom Left	(0,height)
Bottom Right	(width,height)

Performance Tips

Use `shouldRepaint = false` when possible.

Avoid heavy calculations inside `paint()`.

Cache paths when possible.

Use RepaintBoundary for optimization.

Common Use Cases

- Custom backgrounds
 - Wave shapes
 - Chart drawing
 - Game rendering
 - Progress indicators
 - Complex UI shapes
-

Editable Boilerplate Template

You can reuse this template:

```
class MyPainter extends CustomPainter {  
  @override  
  void paint(Canvas canvas, Size size) {  
    final paint = Paint();  
  
    // Your drawing here  
  
  }  
  
  @override  
  bool shouldRepaint(covariant CustomPainter oldDelegate) => false;  
}
```

Summary

CustomPainter gives full control over drawing.

Workflow:

1. Create Paint
 2. Create Path / Shape
 3. Draw using Canvas
 4. Attach using CustomPaint widget
-

Full Source Code (Copy-Paste Ready)

This is the complete reusable boilerplate:

```
import 'package:flutter/material.dart';

class MyCustomPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..color = Colors.blue
      ..style = PaintingStyle.fill
      ..strokeWidth = 2;

    final path = Path()
      ..moveTo(0, size.height)
      ..lineTo(size.width / 2, 0)
      ..lineTo(size.width, size.height)
      ..close();

    canvas.drawPath(path, paint);
  }

  @override
  bool shouldRepaint(CustomPainter oldDelegate) => false;
}

class CustomPaintExample extends StatelessWidget {
  const CustomPaintExample({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: CustomPaint(
          size: const Size(300, 200),
          painter: MyCustomPainter(),
        ),
      ),
    );
  }
}
```

You can edit this document and reuse it for any custom drawing implementation.